BE Final Year Project

Detailed Research Report

Three Project Ideas — Feasibility, Research & Innovation Analysis

Project 1 | AI-Powered Interview Monitoring System	Project 2 | Real Estate Document Verification System	Project 3 | LLM Hallucination Detector

Prepared for BE Final Year Project Selection

Academic Year 2026–27 | Team of 4 Members | ML / GenAI Specialization

PROJECT 1 — AI-POWERED INTERVIEW MONITORING SYSTEM

1.1 What Is This Project? (Simple Explanation)

Imagine a company is hiring 200 people for one job. They cannot talk to all 200 one by one — it takes too long. And even if they do, the interviewer forgets what the first candidate said by the time they reach candidate number 50. On top of that, nowadays many candidates cheat in online interviews — they ask a friend for help, or they type questions into ChatGPT secretly.

Your system solves all three problems at once. It watches the interview like a smart camera, catches any cheating, gives every candidate a score automatically, and even tests whether the candidate knows how to use AI tools properly — because modern companies want employees who can work with AI.

1.2 The Three Core Features Explained

Feature 1: Cheat Detection (Anomaly Detection)

The system watches the candidate's face and eyes the entire time using the computer camera. It checks things like: Are they looking away too many times? Is there a second person in the room? Did they switch to another browser tab? If anything suspicious happens, it flags it and records a timestamp.

This is similar to how exam proctoring works on platforms like ProctorU, but applied to interviews — which is a newer use case.

What it detects	Eye gaze direction, head pose, multiple faces, tab switches, phone usage
Technology used	MediaPipe (Google's free library), OpenCV, JavaScript tab-focus detection
Difficulty	Medium — open source tools exist, you just connect them together
Who does it already	Proctorio, Respondus (for exams only — not interviews)

Feature 2: Auto Scoring and Summary Report

After the interview, instead of the HR person remembering everything, your system gives them a neat report. It recorded everything the candidate said, converted it to text (this is called transcription), and then an AI read that text and gave scores for things like: Did they explain the technical concept correctly? Did they communicate clearly? Were they confident?

The HR manager can open a dashboard and instantly see: Candidate A scored 82/100, Candidate B scored 67/100, and here is a two-paragraph summary of what each person said. This makes hiring 10x faster.

Transcription tool	OpenAI Whisper (free, works in Hindi and Marathi too)
Scoring engine	GPT-4o or Claude API — you give it a rubric and it scores each answer
Output format	JSON per candidate: scores, summary, anomaly flags
Difficulty	Low-Medium — this is a well-known pipeline, just needs good rubric design

Feature 3: AI-Tool Proficiency Chatbot (Your Most Innovative Feature)

This is the feature that makes your project unique. Companies today want employees who can use AI tools well — someone who knows how to write a good prompt for ChatGPT or Claude is more productive than someone who does not. But nobody is testing this skill during interviews.

Your system puts a small chatbot inside the interview. When a candidate is stuck on a coding question or a reasoning question, they can open the chatbot and ask it for help — just like they would in a real job. Your system secretly records every prompt they typed and scores it: Was it a specific, clear prompt? Did they provide context? Did they iterate and improve their question? The interviewer can later see exactly how the candidate used AI — a powerful and completely new signal.

Chatbot backend	Claude API or GPT-4o API with session logging
Prompt quality scoring	LLM judge — rates prompts on specificity, context, iteration quality
What interviewer sees	Full prompt history + a Prompt Quality Score out of 10
Innovation level	High — no existing commercial system offers this feature publicly

1.3 Is It Doable? Honest Assessment

Yes, very much so. This is the most balanced project for a BE team of 4. All the individual components exist as open-source libraries — you are not building anything from scratch at the fundamental level. You are connecting smart pieces together in a new way. With 6 months and 4 people, a fully working prototype is very achievable. If you add a clean UI and good demo, this project will impress both your college jury and any company that sees it.

1.4 What Already Exists? (So You Know the Competition)

The following commercial tools exist but none of them have all three features combined, especially not the AI-proficiency chatbot feature:

HireVue	AI video interview analysis — focuses on facial expressions and speech, expensive, black-box, no AI-tool testing
Mercer Mettl	Online interview platform — has proctoring but no AI proficiency scoring
Talview	Interview recording and AI scoring — no chatbot, no prompt quality analysis
Proctorio / Respondus	Exam proctoring only — not designed for interviews at all
IndicWhisper	Open-source speech recognition fine-tuned for Indian languages — useful for you

1.5 Your Innovation — What Nobody Has Done Before

- Prompt Quality Scoring during live interview — completely new. Define a rubric (clarity, specificity, context provided, iteration) and auto-score it with an LLM judge.

- Unified candidate dashboard combining behavioral signals (cheat score) + content quality (answer score) + AI skill (prompt score) — three dimensions in one view.
- Indian language support — use IndicWhisper to transcribe Hindi/Marathi answers, making your tool usable in Indian SMEs and startups that interview in regional languages.
- Dataset contribution — if you record and label 500+ mock interview sessions (with consent), that is a publishable dataset that does not exist anywhere today.

1.6 Recommended Tech Stack

Frontend	React.js with WebRTC for live video, Chart.js for dashboards
Backend	FastAPI (Python) — fast, modern, easy to integrate ML models
Face / Gaze Detection	MediaPipe Face Mesh + OpenCV for real-time analysis
Speech to Text	OpenAI Whisper (open-source, free, multilingual)
LLM for Scoring	Claude API (claude-sonnet) or GPT-4o via API
Database	PostgreSQL for storing transcripts, scores, session logs
Hosting (demo)	Docker + local server or free-tier AWS/GCP for demo

1.7 Research Papers to Cite

These are real published papers you can reference in your report and presentation:

#	Title	Authors / Year	Venue	Key Contribution
1	Automated Analysis of Multimodal Interviews for Personality Assessment	Escalante et al., 2020	IEEE Trans. Affective Computing	Multimodal interview analysis combining audio, video, and text for personality trait prediction — closest to your scoring pipeline
2	Hirability in the Wild: Analysis of Online Conversational Video Resumes	Escalante et al., 2017	IEEE Trans. Affective Computing	Predicting hirability from video features — baseline for your answer scoring module
3	Robust Face Landmark Estimation Under Occlusion	Amos et al., 2016	IEEE ICCV	Foundation for face mesh detection used in gaze tracking (MediaPipe builds on this lineage)
4	Whisper: Robust Speech Recognition via Large-Scale Weak Supervision	Radford et al. (OpenAI), 2022	ICML 2023	The transcription backbone of your system — cite this whenever you mention Whisper
5	Large Language Models are Zero-Shot Reasoners	Kojima et al., 2022	NeurIPS 2022	Basis for using LLMs as zero-shot scorers — supports your rubric-based scoring approach

6	Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena	Zheng et al., 2023	NeurIPS 2023	Shows LLMs can reliably judge quality of outputs — directly supports your LLM-based answer scoring
7	Online Exam Proctoring Technologies: Educational Innovation or Invasion of Privacy?	Swauger, 2020	MIT Press — First Monday	Foundational paper on proctoring ethics — important for your discussion section on privacy

1.8 Verdict

RECOMMENDED — Strong market need, achievable in 6 months, genuinely novel chatbot feature

Publish target: AIES (AAAI/ACM AI, Ethics & Society) workshop or CODS-COMAD India

PROJECT 2 — AI REAL ESTATE DOCUMENT VERIFICATION SYSTEM

2.1 What Is This Project? (Simple Explanation)

Think about what happens when someone in India buys a flat or a plot of land. They need to submit a huge pile of documents to the government — Sale Deed, Encumbrance Certificate, 7/12 Extract (for Maharashtra), NOC from the housing society, Occupancy Certificate, and more. Most people do not know which documents they need or what format they should be in. They go to the government office, wait in line for hours, and then the officer says: 'This document is wrong' or 'You are missing this certificate.' They have to go back home and come again.

Your system solves this before they even leave the house. A person uploads all their property documents to your website, and your AI checks everything: Are all required documents present? Do the details match across documents (e.g., does the Survey Number in the Sale Deed match the one in the EC)? Are there any expired dates, missing signatures, or formatting errors? It gives them a checklist of what to fix before they visit the government office.

2.2 Core Features

Feature 1: Document Classification

First, the system needs to understand what type of document was uploaded. A 7/12 Extract looks very different from a Sale Deed. Your system uses OCR (Optical Character Recognition) to read the text in the uploaded image or PDF, and then classifies it: 'This is an Encumbrance Certificate from Pune registration office.'

Feature 2: Field Extraction and Validation

Once it knows what document it is, it extracts key fields — Survey Number, Owner Name, Date, Property Area, Taluka, District, etc. — and checks if those fields are filled in correctly. Is the date in DD/MM/YYYY format? Is the owner name spelled consistently across documents? Is the property area stated in both sq ft and sq meter as required?

Feature 3: Cross-Document Consistency Check (Most Valuable Feature)

This is where AI makes a real difference. The Survey Number mentioned in the Sale Deed must match the one in the 7/12 Extract and the Encumbrance Certificate exactly. The owner's name must be consistent everywhere. Your system compares all uploaded documents against each other and raises a flag if anything does not match — giving a specific, plain-English explanation like: 'The Survey Number in your Sale Deed (123/4) does not match the 7/12 Extract (123/5). Please verify with your property advisor.'

2.3 Is It Doable? Honest Assessment

This is the most beginner-friendly of the three projects. The technology is well-established — OCR has been around for years, and modern LLMs are excellent at reading and understanding document text. The challenge is not technical difficulty but domain knowledge: you need to understand which documents are required for different property transaction types in Maharashtra/India. Spending two weeks researching

this domain (talking to a property lawyer or visiting a registration office) will make your system much better.

2.4 What Already Exists?

Google Document AI	Excellent OCR and field extraction — but generic, not trained on Indian real estate docs
AWS Textract	Similar to Google — powerful but no India-specific domain knowledge built in
DigiLocker (India)	Stores official documents digitally but does NOT validate them or check cross-document consistency
Leegality / Signzy	Indian legal document signing and verification for businesses — not for individual citizens
Maharashtra e-Registration	Lets you register documents online but gives no pre-submission validation feedback

The gap is clear: no tool currently helps an individual citizen validate their property documents before submission in a user-friendly way, especially for Maharashtra-specific documents like 7/12 Extract and Property Card.

2.5 Your Innovation — What Nobody Has Done Before

- Maharashtra-specific document corpus — building a labeled dataset of Indian real estate documents (even 500 examples) is a publishable data contribution. No public dataset exists.
- Cross-document semantic matching using embeddings — instead of simple string matching for field comparison, use sentence embeddings to handle spelling variations and abbreviations (e.g., 'Shri Ram Patil' vs 'Ram Patil').
- Explainable error messages in simple language — most systems say 'Error detected.' Yours says 'The survey number on your Sale Deed does not match your 7/12. Go to your tahsildar office with document X to get the corrected 7/12 Extract.'
- Multilingual support — accept documents and give feedback in Hindi and Marathi, making the tool genuinely usable by common people in rural Maharashtra.

2.6 Recommended Tech Stack

Frontend	React.js with file upload, document viewer, and checklist UI
Backend	FastAPI (Python)
OCR Engine	Google Cloud Vision API (free tier: 1000 pages/month) or Tesseract (open-source)
Image Preprocessing	OpenCV — deskewing, noise removal before OCR
LLM for Semantic Check	Claude API or GPT-4o for cross-field consistency and error explanation
Embeddings	sentence-transformers library (all-MiniLM-L6-v2) for field similarity matching
Database	PostgreSQL for storing document metadata and validation results

2.7 Research Papers to Cite

#	Title	Authors / Year	Venue	Key Contribution
1	An Overview of the Tesseract OCR Engine	Smith, R., 2007	ICDAR 2007	The foundational open-source OCR tool — cite as the basis of your text extraction pipeline
2	LayoutLM: Pre-training of Text and Layout for Document Image Understanding	Xu et al. (Microsoft), 2020	KDD 2020	Pre-trained model for document understanding — can be fine-tuned for Indian real estate docs
3	DocFormer: End-to-End Transformer for Document Understanding	Appalaraju et al., 2021	ICCV 2021	Multi-modal transformer for forms and scanned documents — relevant to your field extraction module
4	Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks	Reimers & Gurevych, 2019	EMNLP 2019	The basis of your cross-document semantic field matching using sentence embeddings
5	Indian Document OCR: A Survey of Challenges and Approaches	Sharma et al., 2019	IJCA	Surveys challenges specific to Indian script OCR — important background for Devanagari text in your documents
6	Explainability in AI-powered Document Verification Systems	Arrieta et al., 2020	Information Fusion Journal	Framework for making AI decisions explainable — supports your plain-English error message feature

2.8 Verdict

RECOMMENDED for teams wanting strong social impact and high completion certainty

Publish target: Dataset paper at FIRE (Forum for Information Retrieval Evaluation, India) or ICON (NLP conference India)

PROJECT 3 — LLM HALLUCINATION DETECTOR

3.1 What Is This Project? (Simple Explanation)

ChatGPT, Gemini, and Claude are all incredible tools — but they have one serious problem. Sometimes they confidently make up facts that are completely wrong. This is called 'hallucination.' For example, you ask ChatGPT: 'Which researcher published the first paper on transformer neural networks?' and it might give you a real-sounding name and paper title — but both are completely made up. The AI sounds 100% confident, but it is lying without knowing it.

This is a massive problem in fields like medicine (wrong drug dosages), law (fake case citations), education (fake historical facts), and journalism (invented statistics). Your project builds a system that reads any LLM output and tells you: 'This sentence is probably true. This other sentence is very likely hallucinated — here is why and here is the confidence score.' Think of it as a fact-checker that works in real time.

3.2 Two Approaches — Which One to Pick

Option A: Black-Box Detector (Recommended for Your BE Project)

You only look at the text output — you do not need access to the AI's internal code. This means it works on ChatGPT, Gemini, Claude, or any AI. The idea is clever: if you ask the same question 7-8 times with slightly different phrasing and the AI gives inconsistent answers, then those inconsistent parts are probably hallucinated. Consistent facts stay consistent; made-up facts change each time.

Core idea	Sample LLM output multiple times → check consistency using NLI → low consistency = hallucination
NLI model	DeBERTa-v3-large trained on MNLI — checks if sentence A entails, contradicts, or is neutral to sentence B
Optional add-on	Retrieve top-3 web sources and check if the sentence is supported by them
Difficulty	Medium — main challenge is designing a good evaluation benchmark
GPU needed	No — DeBERTa runs well on CPU, though GPU makes it faster

Option B: White-Box / Grey-Box Detector (Research-Level)

Here you load an open-source model like Llama 3 or Mistral 7B and look inside it — specifically at the 'confidence' the model has for each word it generates (called token probability or entropy). If the model is very uncertain about a word (high entropy), that word is more likely to be hallucinated. This approach is more accurate but requires GPU access and deeper ML knowledge.

Key signal	Token-level entropy — high uncertainty = probable hallucination
Model required	Llama-3-8B or Mistral-7B (open-source, downloadable from HuggingFace)
GPU requirement	Yes — at minimum 16GB VRAM (college GPU lab or Colab A100)

Difficulty	High — this is genuine ML research, not just engineering
Advantage	More accurate and publishable at top venues like ACL, EMNLP

3.3 Is It Doable? Honest Assessment

Option A: Yes, definitely doable for your BE team, especially since you have ML/Python experience. The main technical components (DeBERTa, HuggingFace, API sampling) are well-documented with code examples. The hard part is designing a good evaluation: you need a dataset where you know which sentences are hallucinated and which are not, so you can measure whether your system works.

Option B: Doable with your college GPU lab, but it will take more time. If one person focuses purely on the white-box module while others build the pipeline and UI, it is achievable. The combination of both approaches in one paper is genuinely publishable.

3.4 What Already Exists? (Active Research Area)

SelfCheckGPT (2023)	Samples LLM multiple times, uses NLI for consistency check. Your main baseline. Code is on GitHub.
FACTSCORE (2023)	Breaks output into atomic facts, checks each against Wikipedia. Best for biography-style text.
SAFE — Google (2024)	Uses Google Search to verify each sentence. Very accurate but requires search API.
HaluEval Benchmark	Dataset with labeled hallucinated and non-hallucinated LLM outputs. Use this to test your system.
LM-Polygraph	White-box uncertainty quantification library for LLMs. Ready to use if you go Option B.

Important: None of these tools have been evaluated on Indian language text (Hindi, Marathi) or on Indian legal and medical domains. That is your gap to fill.

3.5 Your Innovation — What Nobody Has Done Before

- Indian domain benchmark — create a hallucination evaluation dataset specifically for Indian legal text (IPC sections, court judgments) or medical text (drug interactions, disease descriptions in Indian context). No such dataset exists publicly.
- Hybrid pipeline — combine three signals: SelfCheckGPT consistency + retrieval-augmented fact checking + LLM self-critique ('Does this sentence contradict established knowledge?'). Compare each alone vs the combination. The hybrid comparison is a publishable result.
- Hindi/Marathi hallucination detection — test whether the same LLM hallucinates more in regional Indian languages than in English. This cross-lingual analysis has not been done comprehensively.
- Deployable browser extension — package your detector as a Chrome extension that highlights hallucinated sentences on any webpage where LLM output appears. This gives you a real product, not just a research demo.

3.6 Recommended Tech Stack

Core language	Python — for all ML components
NLI Model	DeBERTa-v3-large via HuggingFace Transformers library
LLM sampling	OpenAI API or Claude API (for black-box consistency sampling)
White-box (Option B)	Llama-3-8B via HuggingFace, LM-Polygraph library for entropy extraction
Retrieval	Tavily API (free tier) or SerpAPI for web-based fact checking
Evaluation	HaluEval dataset + your custom Indian domain dataset
UI / Demo	Streamlit for quick demo, or React for polished frontend
Browser Extension	Chrome Extension API with JavaScript calling your FastAPI backend

3.7 Research Papers to Cite

#	Title	Authors / Year	Venue	Key Contribution
1	SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models	Manakul et al., 2023	EMNLP 2023	Your primary baseline method — consistency sampling with NLI. Must cite.
2	FActScoring: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation	Min et al., 2023	EMNLP 2023	Atomic fact decomposition approach — compare your method against this
3	HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models	Li et al., 2023	EMNLP 2023	Evaluation dataset to test your detector — download and use this directly
4	SAFE: Improving Factuality of Language Models via System-Aware Factual Evaluation	Wei et al. (Google), 2024	arXiv 2024	Search-augmented fact checking — basis for your retrieval-augmented component
5	Language Models (Mostly) Know What They Know	Kadavath et al. (Anthropic), 2022	arXiv 2022	Shows LLMs have calibrated self-knowledge — basis for your LLM self-critique component
6	DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing	He et al. (Microsoft), 2023	ICLR 2023	The NLI backbone model you will use for entailment checking. Must cite.
7	Measuring and Mitigating Hallucinations in Large Language Models: A Survey	Ji et al., 2023	ACM Computing Surveys	Comprehensive survey of all hallucination detection methods — essential background reading and citation

8	LLM-Check: Investigating Detection of Hallucinations in Large Language Models	Khaliq et al., 2024	arXiv 2024	Recent comparison of hallucination detection methods — shows where gaps exist and helps position your work

3.8 Verdict

HIGH-RISK, HIGH-REWARD — Best for teams with strong ML depth and access to GPU	Publish target: ACL Findings, EMNLP 2025 Workshop on Faithfulness in NLP, or FIRE India